

ESP32forth and WiFi interface

Marc PETREMANN



Version 1.0 - May 18, 2026

Table des matières

Preamble.....	3
WiFi Vocabulary Extension.....	3
ESP32 and its WiFi interface.....	5
Steps for a complete connection.....	6
Station mode initialization.....	6
Index lexical.....	7

Preamble

This manual is intended for getting started with the WiFi interface for ESP32forth,

WiFi Vocabulary Extension

Some words have been added to the WiFi vocabulary. Here is the extended version of this vocabulary to be used in the ESP32forth.ino source file:

```
# define OPTIONAL_WIFI_SUPPORT \
/* WiFi */ \
XV(WiFi, "WiFi.config", WIFI_CONFIG, \
    WiFi.config(ToIP(n3), ToIP(n2), ToIP(n1), ToIP(n0)); DROPn(4)) \
XV(WiFi, "WiFi.begin", WIFI_BEGIN, WiFi.begin(c1, c0); DROPn(2)) \
XV(WiFi, "WiFi.disconnect", WIFI_DISCONNECT, WiFi.disconnect(true)) \
XV(WiFi, "WiFi.status", WIFI_STATUS, PUSH WiFi.status()) \
XV(WiFi, "WiFi.macAddress", WIFI_MAC_ADDRESS, WiFi.macAddress(b0); DROP) \
XV(WiFi, "WiFi.localIP", WIFI_LOCAL_IPS, PUSH FromIP(WiFi.localIP())) \
XV(WiFi, "WiFi.mode", WIFI_MODE, WiFi.mode((wifi_mode_t) n0); DROP) \
XV(WiFi, "WiFi.setTxPower", WIFI_SET_TX_POWER, \
WiFi.setTxPower((wifi_power_t) n0); DROP) \
XV(WiFi, "WiFi.getTxPower", WIFI_GET_TX_POWER, PUSH WiFi.getTxPower()) \
XV(WiFi, "WiFi.softAP", WIFI_SOFTAP, n0 = WiFi.softAP(c1, c0); NIP) \
XV(WiFi, "WiFi.softAPIP", WIFI_SOFTAP_IP, PUSH FromIP(WiFi.softAPIP())) \
XV(WiFi, "WiFi.softAPBroadcastIP", WIFI_SOFTAP_BROADCASTIP, PUSH \
FromIP(WiFi.softAPBroadcastIP())) \
XV(WiFi, "WiFi.softAPNetworkID", WIFI_SOFTAP_NETWORKID, PUSH \
FromIP(WiFi.softAPNetworkID())) \
XV(WiFi, "WiFi.softAPConfig", WIFI_SOFTAP_CONFIG, n0 = \
WiFi.softAPConfig(ToIP(n2), ToIP(n1), ToIP(n0))) \
XV(WiFi, "WiFi.softAPdisconnect", WIFI_SOFTAP_DISCONNECT, n0 = \
WiFi.softAPdisconnect(n0)) \
XV(WiFi, "WiFi.softAPgetStationNum", WIFI_SOFTAP_GET_STATION_NUM, PUSH \
WiFi.softAPgetStationNum()) \
/* WiFi scan */ \
/* Récupère le nombre de réseaux WiFi détectés */ \
XV(WiFi, "WiFi.scanNetworks", WIFI_SCANNETWORKS, PUSH \
(cell_t)WiFi.scanNetworks()) \
/* Récupère le SSID du réseau indexé par n0 */ \
XV(WiFi, "WiFi.SSID", WIFI_SSID, { String s = WiFi.SSID(n0); n0 = (cell_t) \
strdup(s.c_str()); }) \
/* Récupère la puissance (RSSI) du réseau indexé par n0 */ \
XV(WiFi, "WiFi.RSSI", WIFI_RSSI, n0 = \
(cell_t)WiFi.RSSI(n0)) \
/* Récupère le canal (channel) du réseau indexé par n0 */ \
XV(WiFi, "WiFi.channel", WIFI_CHANNEL, n0 = \
(cell_t)WiFi.channel(n0)) \
/* Récupère le type de cryptage du réseau indexé par n0 */ \
```

```
XV(WiFi, "WiFi.encryptionType", WIFI_ENCRYPTIONTYPE, n0 =  
(cell_t)WiFi.encryptionType(n0)) \  
/* Efface les résultats du scan de la mémoire */ \  
XV(WiFi, "WiFi.scanDelete", WIFI_SCANDELETE, WiFi.scanDelete())
```

ESP32 and its WiFi interface

The ESP32's WiFi interface is a powerful subsystem that allows the microcontroller to communicate wirelessly via the TCP/IP stack. In **ESP32forth**, this interface is driven by words that bridge the gap with native C++ libraries. It primarily operates in three modes: **Station (STA)** to connect to a router, **Access Point (AP) to create its own network**, or the **hybrid AP+STA** mode .

The process follows a sequential logic: first, the mode is defined with ``WiFi.mode`` , then the connection or service is initiated with ``WiFi.begin`` or ``WiFi.softAP`` . Since the connection is **asynchronous** , the program must monitor the status via ``WiFi.status`` before exchanging data. Once connected, the ESP32 obtains an IP address (retrievable with ``WiFi.localIP``) and can open sockets **to** act as a web client or server, thus allowing remote control of hardware via a simple browser or HTTP requests.

WiFi 4 (Standard 2.4 GHz) is the standard for the vast majority of projects.

- **ESP32 Original (WROOM/WROVER Series)** : The most common. WiFi + Bluetooth Classic + BLE.
- **ESP32-S2** : WiFi only (no Bluetooth). Very energy efficient.
- **ESP32-S3** : WiFi + Bluetooth LE. Very powerful, ideal for AI and displays.
- **ESP32-C3** : WiFi + Bluetooth LE. Single-core cost-effective model (RISC-V architecture).

The "Next Gen" (WiFi 6) models are more recent (2024-2026) and offer better management of congested networks:

- **ESP32-C6** : WiFi 6 + Bluetooth 5 + Zigbee/Thread (for "Matter" home automation).
- **ESP32-C5** : This is the first **Dual-Band model** (2.4 GHz and **5 GHz**).

The exception (Without WiFi), a recent series is an exception:

- **ESP32-P4** : This board is ultra-powerful (400 MHz) and has video capabilities, but it lacks built-in **wireless connectivity** . It is intended for industrial applications or local interfaces.

How do I identify the WiFi interface on my card?

Look at the metal module (often silver) on your card: if you see an "S" or "zig-zag" shaped line on the circuit board or a small ceramic piece at one end, that's the WiFi antenna.



The **ESP-NOW protocol** allows communication between ESP32 boards via WiFi without requiring a connection to a router.

Steps for a complete connection

Launching `WiFi.begin` is not enough to guarantee that you are connected. The process is asynchronous. Here's how to structure a clean connection:

Station mode initialization

Before connecting, it is crucial to tell the ESP32 to behave like a client (Station) and not like an access point.

```
WiFi.disconnect \ Optional: clears old sessions
1 WiFi.mode    \ 1 = Station Mode (STA)
```

Once `WiFi.begin` is launched, you need to monitor the status.

```
z" SSID_NETWORK" z" PASSWORD" WiFi.begin

: wait-wifi ( -- )
  begin
    WiFi.status 3 <> \ 3 corresponds to WL_CONNECTED
  while
    ." Waiting for WiFi..." cr
    1000 ms
  repeat
    ." Connected! IP: " WiFi.localIP .ip cr ;
```

Lexical index

